Assignment 3: Association Rule Mining for Market Basket Analysis

Data Analytics 1 — MovieLens recommender built on single-antecedent rules

Team 2024101035_2024102017

October 14, 2026

Abstract

We turn MovieLens ratings into "liked-movie" baskets and mine association rules of the form $\{A\} \to Y$ with an **FP-growth implementation written from scratch**. The rules are then used to recommend movies to held-out users. A from-scratch Apriori returns identical itemsets but is up to 6.4× slower on these dense baskets, which supports the choice of FP-growth. A study of minsup and minconf on a validation split selects **minsup = 0.10** and **minconf = 0.50**, giving 21 026 frequent itemsets and 9 281 rules. Candidate movies are ranked by the highest lift among the rules that fire. On the 122 test users, evaluated with a chronological half split, the recommender reaches precision@10 = 0.209 and recall@10 = 0.061. A popularity baseline reaches 0.175 and 0.049; the recommender is about 19 % better on precision@10 and 25 % better on recall@10.

TA clarifications followed. (1) Both mining algorithms are implemented from scratch; no `mlxtend` or similar library is used (Sec. 2). (2) This report is a PDF. (3) For Part C.2 every test user's liked movies are sorted by timestamp, so the older half is the input and the newer half is the ground truth (Sec. 5). (4) The score used to choose the top-k recommendations is stated and justified (Sec. 6). All numbers below come from the executed notebook `2024101035_2024102017_recommender.ipynb`, which uses a single random seed (42).

1 Data preprocessing (Part A)

Dataset. We use MovieLens *ml-latest-small*, the 100K-rating release that contains `ratings.csv` and `movies.csv`. It has 100 836 ratings from 610 users on 9 724 movies.

A.1 Binarisation and filtering. A rating ≥ 3.0 is treated as a positive interaction ("liked") and every other rating is dropped. Each (user, movie) pair therefore becomes a binary liked flag. Users with fewer than 5 liked movies are then removed, so that every basket can be split into a meaningful input half and ground-truth half. There were no duplicate (user, movie) pairs.

Step	Ratings	Users	Movies
Raw ratings	100 836	610	9 724
Keep rating ≥ 3.0	81 763	609	8 452
Keep users with ≥ 5 likes	81 759	608	8 452

Table 1: Effect of each preprocessing step. The activity filter removes only one user, because MovieLens already requires at least 20 ratings per user.

A.2 Transactions. Grouping by `userId` gives 608 transactions. Each one is the set of unique `movieId`s the user liked; titles are attached only for display. We also keep each user's liked movies as a list sorted by timestamp (ties broken by `movieId`), because Part C.2 needs it. Basket sizes are heavy-tailed: the mean is 134.5, the median 61.5, the minimum 10 and the maximum 2 117 (Fig. 1).

A.3 Split. Users, not individual ratings, are shuffled with seed 42 and split 80/20 into **486 training** and **122 test** transactions. A test user's whole basket is therefore unseen during mining. To tune thresholds without touching the test set, the training users are split once more, 80/20, into 389 inner-training and 97 validation users.

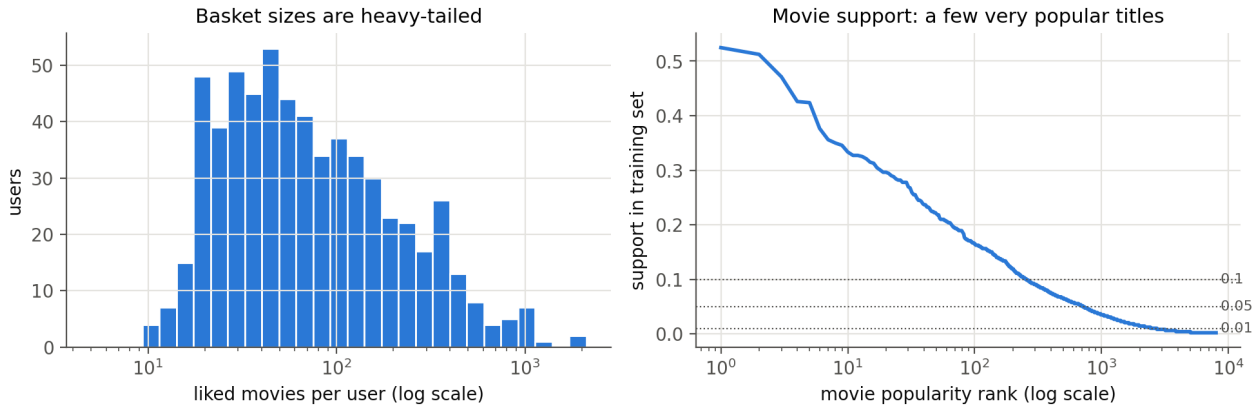


Figure 1: Left: liked movies per user. Right: support of each movie in the training set, by popularity rank. Only 2 movies are liked by more than 50 % of users, about 250 reach 10 %, and about 2 400 reach 1 %. This density, with many popular movies and long baskets, drives every later design decision.

2 Association rule mining (Part B.1)

2.1 Choice of algorithm: FP-growth

The baskets are long and dense. The median user likes about 60 movies, and over 250 movies are liked by more than 10 % of users. This is the worst case for Apriori, for two reasons.

- It makes one full database scan per itemset length, and every scan tests every candidate against every basket.
- Its candidate sets are huge: at minsup 0.08 there are already 6 182 frequent pairs and more than 50 000 frequent 4-itemsets.

FP-growth compresses the database once into a prefix tree. Popular movies sit near the root and share prefixes, so the tree is compact precisely when the data is dense. It then grows patterns recursively from conditional trees without generating any candidates. We therefore use **FP-growth** as the miner and implement Apriori only as a correctness check and runtime baseline.

Implementation (from scratch).

- `build_fp_tree` counts item frequencies, drops infrequent items and inserts each transaction in descending-frequency order. It also maintains a header table of node-links.
- `fp_growth` walks the header table from the least to the most frequent item. For each item it emits suffix $\cup \{\text{item}\}$, builds the item's conditional pattern base (the prefix paths weighted by node counts) and recurses.
- A *single-path shortcut* handles conditional trees that are one chain: every combination of a chain's nodes is emitted directly. This matters on dense data, where deep conditional trees are often chains.
- `apriori` is the textbook level-wise algorithm: join two $(k-1)$-itemsets that share a prefix, prune candidates with an infrequent subset, and count the survivors with one scan of the transactions.

Correctness and runtime. Both miners were run on the 486 training baskets for minsup 0.30 down to 0.08. At every threshold they returned *exactly the same* itemsets and counts. FP-growth

becomes steadily faster in relative terms as minsup falls (Table 2, Fig. 2), reaching a 6.4× speed-up at minsup 0.08.

minsup	frequent itemsets	FP-growth (s)	Apriori (s)	speed-up	identical
0.20	186	0.04	0.10	2.7×	yes
0.15	1 105	0.10	0.45	4.3×	yes
0.12	4 818	0.35	1.36	3.8×	yes
0.10	21 026	0.88	4.86	5.5×	yes
0.09	53 987	1.45	8.86	6.1×	yes
0.08	169 506	3.51	22.36	6.4×	yes

Table 2: Runtime of the two from-scratch miners on the training set (single run each, wall-clock seconds).

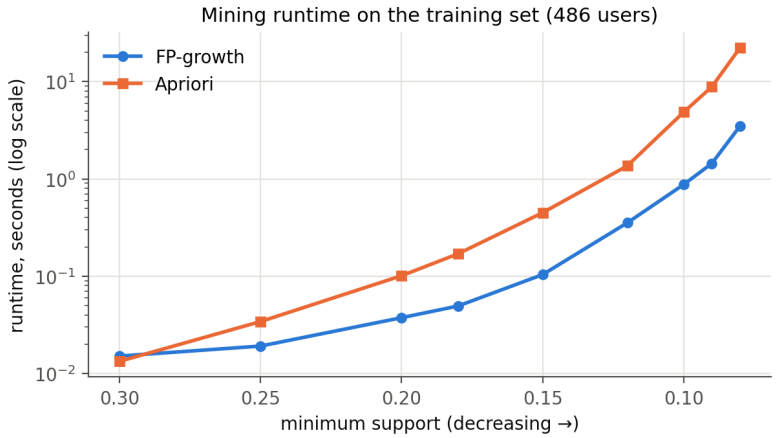


Figure 2: Mining runtime against minsup (log scale). The gap between the two miners widens as the itemset count grows.

2.2 Rule generation: $\{A\} \to Y$

For every frequent itemset I with $|I| \geq 2$ and every movie $a \in I$ we form the rule $\{a\} \to I \setminus \{a\}$, with

$$\text{sup} = s(I), \qquad \text{conf} = \frac{s(I)}{s(\{a\})}, \qquad \text{lift} = \frac{\text{conf}}{s(I \setminus \{a\})}.$$

By downward closure every subset of I is frequent, so all three quantities are looked up in the mined dictionary without another database scan. This produces *all* single-antecedent rules, with consequents of any size, as the assignment requires.

3 Threshold experiments (Part B.2)

Why minsup does not go down to 0.01. The suggested range was $[0.01, 0.1]$. On this data the frequent-itemset count roughly triples for every 0.01 decrease below 0.12: from 4 818 at 0.12 to 21 026 at 0.10 and 516 843 at 0.07 (2.6 million rules). A log-linear fit predicts about 2.8×10^6 itemsets at 0.05 and about 10^8 at 0.01 (Fig. 3, left). The cause is that a few hundred users each like hundreds of the same popular films, so very long itemsets stay frequent. Such rule sets cannot be stored, ranked or inspected, and, as shown below, they bring no gain in recommendation quality. We therefore search minsup $\in \{0.25, 0.20, 0.18, 0.15, 0.12, 0.10, 0.09, 0.08, 0.07\}$ and minconf $\in \{0.1, \ldots, 0.8\}$. The minconf range extends past 0.5 because, as shown below, minconf only starts to matter above 0.5.

What was measured. For every (minsup, minconf) pair we recorded:

- on the full training set: the number of frequent itemsets, the number of rules and the mining time;
- on the validation users, after mining the 389 inner-training users: precision@10, recall@10, F1@10 and coverage, with the exact protocol of Part C. Coverage is the share of users who receive a full list of 10 recommendations.

In this study recommendations are ranked by `max_conf` (Sec. 6). That score depends only on single-movie rules, so it is cheap to evaluate even at minsup 0.07, where there are 2.6 million rules. The ranking score itself is chosen afterwards, at the chosen thresholds.

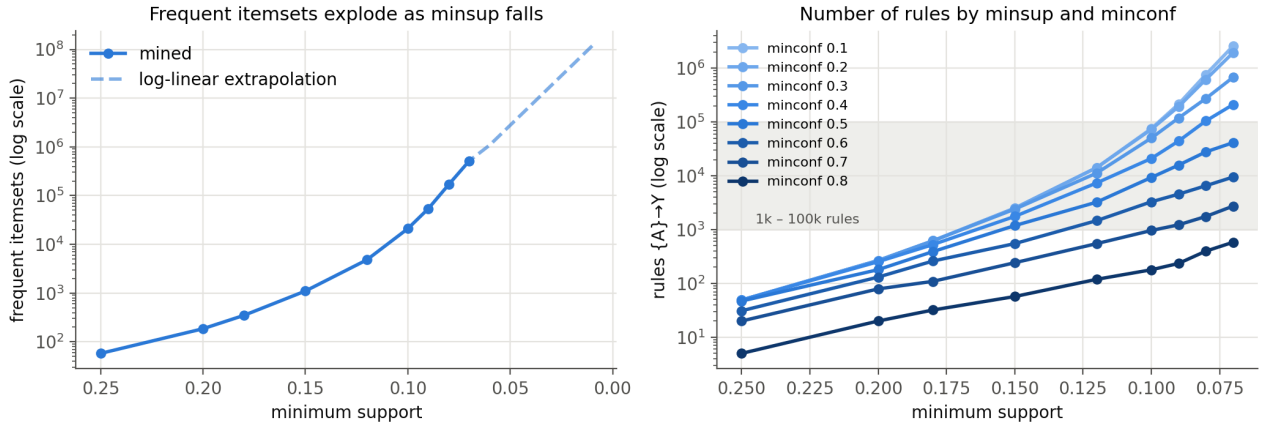


Figure 3: Left: frequent itemsets against minsup, mined (solid) and extrapolated (dashed). Right: number of rules for each minconf; the shaded band is 1k–100k rules.

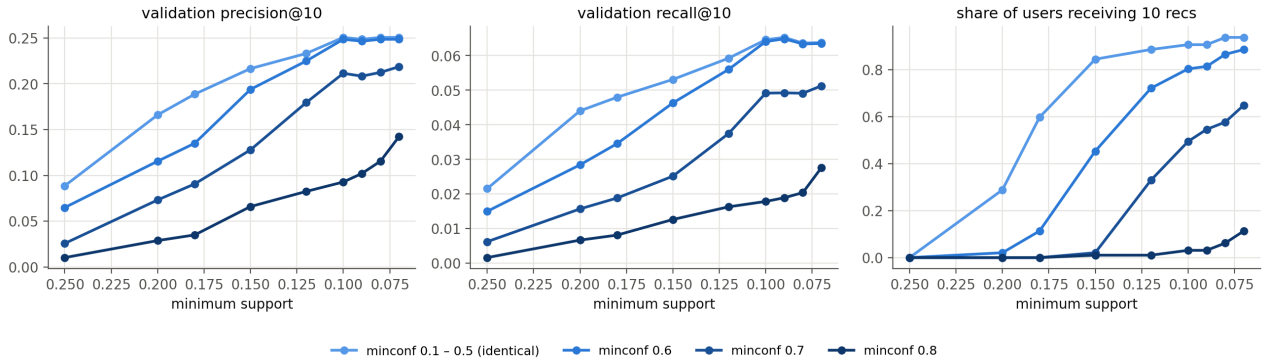


Figure 4: Validation precision@10, recall@10 and coverage against minsup. The curves for minconf 0.1–0.5 coincide exactly and are drawn as one line.

Findings.

1. **Quality saturates at minsup ≈ 0.10.** Validation precision@10 rises from 0.09 (minsup 0.25) to 0.25 (minsup 0.10), then stays flat at 0.249–0.251 down to 0.07. Over that last stretch the rule count grows about 35× (at minconf 0.1). Lower minsup adds rules about rarer movies, but the rarer movies almost never make it into a user's top 10.
2. **minconf ≤ 0.5 changes the rule count but not the recommendations.** At minsup 0.10, raising minconf from 0.1 to 0.5 cuts the rules from 74 986 to 9 281. The validation curves stay identical. Of the 65 705 rules removed, 62 962 have multi-movie consequents and 2 743 have a single movie. Since precision, recall and coverage are all unchanged, none of the removed rules ever puts a movie into a top-10 list. A low-confidence rule $\{a\} \to \{y\}$ usually comes from a very popular antecedent a, and y is already reached through stronger rules.

3. **minconf ≥ 0.6 starts to hurt.** At 0.6 coverage drops from 91 % to 80 %, because users whose movies have only weaker rules get short lists. At 0.7 and 0.8 precision and recall collapse as well.

Selection rule and final choice. We apply the following steps, encoded in the notebook.

(i) Keep settings that produce a *meaningful number of rules*: between 1 000 and 100 000.

(ii) Among them, find the best validation F1@10. Settings within 1 % of the best count as ties, since a 1 % difference is about one hit across 97 users.

(iii) Among the ties, keep those whose coverage is within one percentage point of the best.

(iv) Prefer the largest minconf, then the largest minsup; this gives the smallest rule set built from the most reliable rules.

The result is **minsup = 0.10, minconf = 0.50**.

minsup	minconf	rules	val P@10	val R@10	val F1@10	coverage
0.12	0.5	3 230	0.233	0.059	0.095	0.887
0.10	0.4	20 918	0.251	0.065	0.103	0.907
0.10	**0.5**	**9 281**	**0.251**	**0.065**	**0.103**	**0.907**
0.10	0.6	3 280	0.249	0.064	0.102	0.804
0.09	0.5	15 775	0.249	0.065	0.103	0.907

Table 3: Settings near the chosen one. minsup 0.09 gives the same quality with 70 % more rules. minconf 0.6 loses 10 points of coverage, and minsup 0.12 loses precision.

The final rule set, mined on all 486 training users, contains 21 026 frequent itemsets and **9 281 rules** with 194 distinct antecedents. By consequent size there are 2 893 rules with one movie, 3 953 with two, 1 914 with three, 476 with four, 44 with five and 1 with six. Support ranges from 0.101 to 0.364, confidence from 0.50 to 0.98 (median 0.565), and lift from 1.03 to 5.64 (median 2.69). Every rule has lift > 1, so every rule describes a positive association.

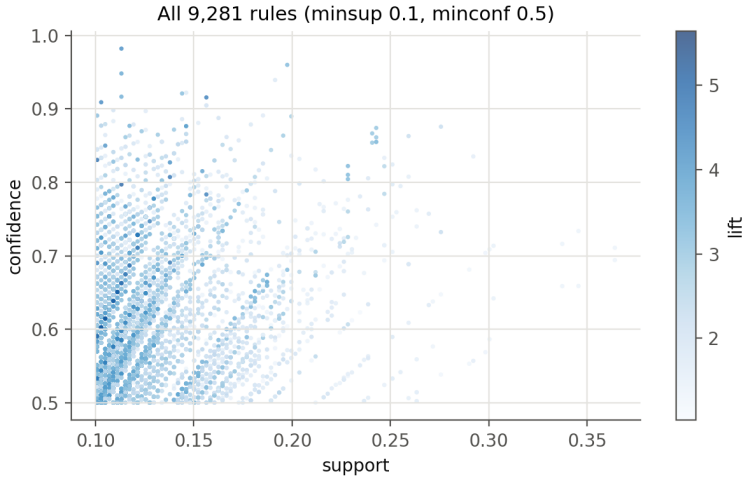


Figure 5: Support against confidence for all 9 281 rules, coloured by lift. The highest-lift rules sit at low support, and the highest-support rules have only modest lift.

4 Top-100 rule lists (Part C.1)

The 9 281 rules are ranked two ways. *By support*, ties are broken by confidence and then lift; this list is in `_top100RulesBySup.txt`. *By confidence*, ties are broken by support and then lift; this list is in `_top100RulesByConf.txt`.

By support the list is dominated by the most popular titles of the mid-1990s. The top rules are $\{Shawshank\ Redemption\} \to \{Forrest\ Gump\}$ (sup 0.364, conf 0.71) and $\{Pulp\ Fiction\} \to \{Forrest\ Gump\}$ (sup 0.346, conf 0.73). Lift is low, 1.35–1.40: these films co-occur mainly because each is liked by almost everyone. Support in this list ranges from 0.226 to 0.364, and 17 of the 100 rules have two-movie consequents.

By confidence the list is led by *franchises and sequels.* Examples are $\{The\ Dark\ Knight\ Rises\} \to \{The\ Dark\ Knight\}$ (conf 0.98, lift 4.26), $\{The\ Godfather\ Part\ II\} \to \{The\ Godfather\}$ (0.96), $\{Back\ to\ the\ Future\ Part\ II\} \to \{Back\ to\ the\ Future\}$ (0.95) and $\{Kill\ Bill\ Vol.\ 2\} \to \{Kill\ Bill\ Vol.\ 1\}$ (0.92, lift 4.59). These rules point from the later or less popular title to the original: almost everyone who liked a sequel also liked the original, but not the other way round. The rest of the list points into a few universally liked films, for example $\{Mrs.\ Doubtfire\} \to \{Forrest\ Gump\}$ (0.94, lift only 1.79). Confidence in this list ranges from 0.83 to 0.98 and support from 0.10 to 0.29.

Rules in both lists. Exactly **10 rules** appear in both top-100 lists (Table 4). They are the rare rules that are both very common and very reliable: the Star Wars and Lord of the Rings trilogies, plus two blockbuster rules into *Forrest Gump*. The small overlap shows that support and confidence capture different things. High support means the films are popular; high confidence means one film strongly implies the other.

Sup. rank	Conf. rank	Rule	Support	Conf.	Lift
13	90	$\{Jurassic\ Park\ (1993)\} \to \{Forrest\ Gump\ (1994)\}$	0.292	0.835	1.59
17	30	$\{Star\ Wars\ V:\ Empire\ Strikes\ Back\} \to \{Star\ Wars\ IV:\ A\ New\ Hope\}$	0.276	0.876	2.33
32	42	$\{Star\ Wars\ VI:\ Return\ of\ the\ Jedi\} \to \{Star\ Wars\ IV:\ A\ New\ Hope\}$	0.259	0.863	2.29
48	91	$\{Apollo\ 13\ (1995)\} \to \{Forrest\ Gump\ (1994)\}$	0.247	0.833	1.59
55	31	$\{LOTR:\ The\ Two\ Towers\} \to \{LOTR:\ Fellowship\ of\ the\ Ring\}$	0.243	0.874	3.08
56	45	$\{LOTR:\ Return\ of\ the\ King\} \to \{LOTR:\ Fellowship\ of\ the\ Ring\}$	0.243	0.861	3.03
57	53	$\{LOTR:\ Fellowship\ of\ the\ Ring\} \to \{LOTR:\ The\ Two\ Towers\}$	0.243	0.855	3.08
58	54	$\{LOTR:\ Fellowship\ of\ the\ Ring\} \to \{LOTR:\ Return\ of\ the\ King\}$	0.243	0.855	3.03
60	35	$\{LOTR:\ The\ Two\ Towers\} \to \{LOTR:\ Return\ of\ the\ King\}$	0.241	0.867	3.07
61	56	$\{LOTR:\ Return\ of\ the\ King\} \to \{LOTR:\ The\ Two\ Towers\}$	0.241	0.854	3.07

Table 4: The 10 rules present in both the top-100-by-support and the top-100-by-confidence lists.

5 Evaluation logic (Part C.2)

For every test user with more than one liked movie (all 122 test users qualify, since each has at least 10):

1. **Chronological split.** The liked movies are sorted by rating timestamp, oldest first. The first $\lfloor n/2 \rfloor$ form the *input* (the user's watch history) and the remaining $\lceil n/2 \rceil$ form the *ground truth.* Older viewing therefore motivates newer recommendations, which mirrors real use and avoids leaking future preferences into the input.
2. **Firing rules.** Every rule $\{a\} \to Y$ with a in the input fires. Each movie $y \in Y$ that is not already in the input becomes a candidate.

3. **Ranking.** Each candidate is scored over all fired rules that contain it (Sec. 6). Ties are broken by the movie's training-set popularity and then by `movieId`, and the top k are recommended.

4. **Metrics.** With R_k the top-k list and G the ground truth, precision@k $= |R_k \cap G|/k$ and recall@k $= |R_k \cap G|/|G|$. A user who receives fewer than k recommendations still has k as the precision denominator, so an empty list counts as zero rather than being skipped. Both metrics are averaged over all evaluated users.

To make recommendation fast, the fired-rule information is precomputed once into a table mapping each antecedent to its candidates and their aggregated confidence and lift. A recommendation then costs one dictionary lookup per input movie.

6 Choosing the top-k: the ranking score (Part C.3 clarification)

Three scores were compared *on the validation users*, never on the test set, using rules mined on the inner-training users at the chosen thresholds.

- `max_conf`: the highest confidence of any fired rule recommending y, i.e. the best single estimate of $P(\text{likes } y \mid \text{liked } a)$.
- `sum_conf`: the sum of the confidences of all fired rules recommending y. This is a voting score: a movie implied by many watched films scores higher.
- `max_lift`: the highest lift of any fired rule recommending y. Lift measures how much more likely y becomes *given* a, compared with its overall popularity.

One property of `max_conf` follows from the definitions and was verified in code: for every $y \in Y$, $\text{conf}(\{a\} \to \{y\}) \geq \text{conf}(\{a\} \to Y)$, and $\{a\} \to \{y\}$ is itself a rule above minconf. Multi-movie consequents therefore never change a `max_conf` ranking. They do matter for `sum_conf` (more votes) and for `max_lift`. Lift divides by the support of the whole consequent, so a rule with a rarer multi-movie consequent can have higher lift than its single-movie sub-rule; this happens for 15 723 (antecedent, movie) pairs in the final rule set.

Score (validation)	P@1	P@5	P@10	R@5	R@10
`max_conf`	0.351	0.270	0.251	0.035	0.065
`sum_conf`	0.381	0.272	0.239	0.037	0.063
`max_lift`	**0.392**	**0.291**	**0.268**	0.036	**0.070**
popularity baseline	0.289	0.204	0.190	0.025	0.047

Table 5: Ranking scores compared on the 97 validation users.

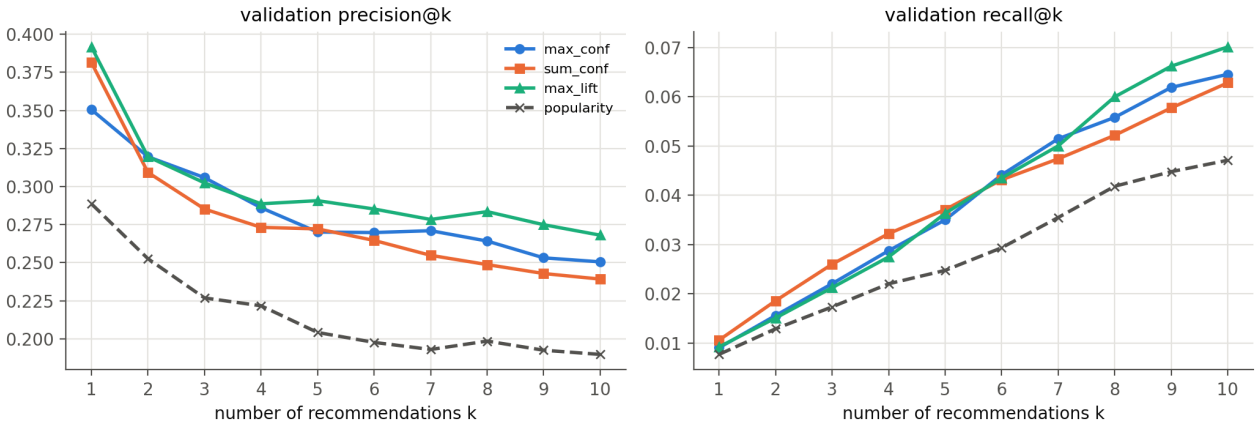


Figure 6: Validation precision@k and recall@k for the three ranking scores and the popularity baseline.

Chosen score: `max_lift`. It gives the best precision@10 and recall@10 on validation, and it is the most sensible score for this data. Confidence is biased towards blockbusters. *Forrest Gump* is liked by 52 % of users, so rules into it are confident almost automatically: 23 of the top 100 rules by confidence point to it, with lift only about 1.6–1.8. Confidence-based scores therefore keep recommending the same handful of films to everyone. Lift divides out that base popularity and ranks highest the movies that are *specifically* implied by what this user watched. A sequel after its original has lift 3.3–4.6, for example, while a rule into another blockbuster has lift about 1.4. The differences between the three scores are small (97 users). `max_lift` leads on precision for $k \geq 5$ and on recall for $k \geq 8$, `sum_conf` is slightly ahead on recall for small k, and all three clearly beat popularity.

7 Results: precision and recall for $k = 1 \ldots 10$ (Part C.3)

The final rules, mined on all 486 training users, are applied with `max_lift` to the 122 test users.

k	Precision (rules)	Recall (rules)	Precision (popularity)	Recall (popularity)
1	0.221	0.008	0.213	0.005
2	0.221	0.014	0.209	0.012
3	0.221	0.020	0.202	0.017
4	0.223	0.027	0.193	0.021
5	0.218	0.033	0.184	0.026
6	0.213	0.039	0.182	0.031
7	0.220	0.046	0.186	0.036
8	0.221	0.052	0.186	0.040
9	0.217	0.057	0.181	0.045
10	0.209	0.061	0.175	0.049

Table 6: Average precision@k and recall@k on the 122 test users.

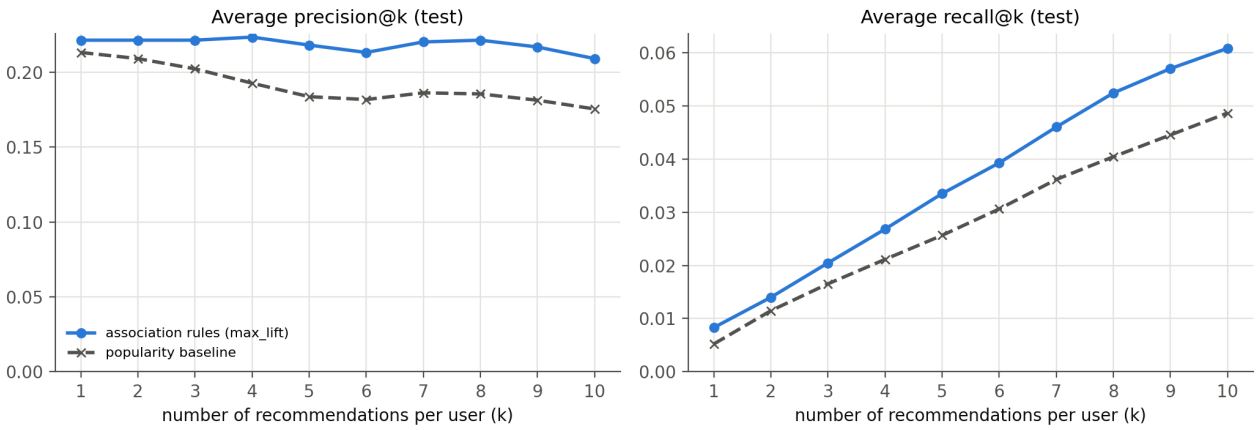


Figure 7: Average precision@k (left) and recall@k (right) on the test set, as the number of recommendations per user varies from 1 to 10.

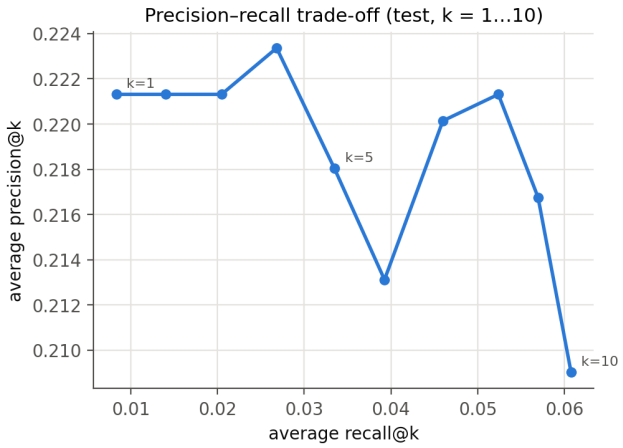


Figure 8: Precision–recall trade-off as k grows from 1 to 10.

Interpretation.

- **Precision is almost flat**, between 0.21 and 0.22 for every k. On average about one in five recommended movies is one the user goes on to like. Recommendations further down the list are nearly as good as the first one, so the score orders candidates well but no candidate is a near-certain hit.
- **Recall grows almost linearly** with k, from 0.008 to 0.061, because each extra recommendation adds hits at a nearly constant rate. The absolute values are low because the ground truth is large: a test user has 58.9 movies to predict on average (median 29), and only 10 recommendations are allowed. Even a perfect recommender would reach only **0.390** average recall@10 here.
- **The rules can only ever recommend 194 movies**, those that appear in some rule. On average only **32 %** of a user's ground-truth movies are recommendable at all; the rest are too rare to pass minsup = 0.10. This is the main structural limit of support-based rules on a long-tailed catalogue.
- **The rules beat the popularity baseline at every** k: 0.209 against 0.175 precision@10 and 0.061 against 0.049 recall@10. The gap is within noise at $k = 1$ (0.221 against 0.213) and widens with k: popularity runs out of good generic titles, while the lift-ranked rules keep finding candidates specific to each user.
- **Coverage:** 115 of the 122 test users (94 %) receive a full list of 10, one receives a shorter list, and 6 receive none, because none of their early movies is an antecedent of any rule. 69.7 % of users get at least one hit in their top 10.

8 Sample users (Part C.4)

Four test users were picked by fixed criteria rather than by hand: the best recall@10, a typical user (median precision@10 among users with at least one hit), the user with the largest history, and the user with the smallest history.

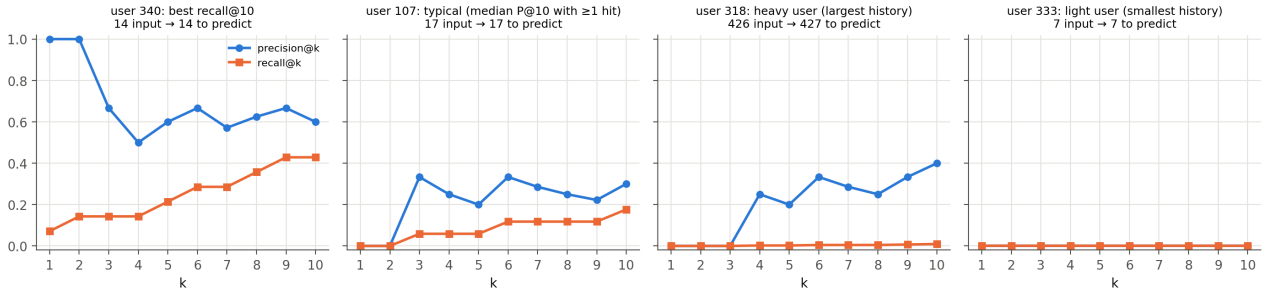


Figure 9: Precision@k and recall@k for four sample test users.

- **User 340 (best case; 14 input → 14 to predict).** The history is mid-1990s mainstream: *Stargate*, *Clear and Present Danger*, *Forrest Gump*. The first two recommendations, *The Firm* and *Crimson Tide*, are both hits, giving precision 1.0 at $k = 1, 2$. At $k = 10$ precision is 0.6 and recall 0.43, with hits including *Braveheart*, *Terminator 2* and *Jurassic Park*. This user's taste matches the dense core of the dataset, where the rules are strongest.
- **User 107 (typical; 17 → 17).** The tastes are similar but the user is less predictable. The first two recommendations miss; hits appear at $k = 3$ (*Crimson Tide*) and later (*Braveheart*, *Stargate*). Precision settles around 0.3 and recall reaches 0.18 at $k = 10$, which is the typical pattern: a few correct titles mixed with reasonable but unwatched blockbusters.
- **User 318 (heavy user; 426 → 427).** With a huge input most rules fire, and the remaining candidates are popular films this user had not yet rated. Precision reaches 0.4 at $k = 10$ (*Shawshank Redemption*, *Se7en*, *Reservoir Dogs*, *Up*). Recall stays near 0.01, because 10 recommendations can never cover 427 future likes. For heavy users recall@10 is not a meaningful measure.
- **User 333 (light user; 7 → 7).** The input mixes older dramas (*What's Eating Gilbert Grape*, *Patton*) with early-2000s comedies (*Small Time Crooks*, *Shanghai Noon*, *Scary Movie*). Of the 7 input movies only one, *One Flew Over the Cuckoo's Nest*, is the antecedent of any rule; the other six are below minsup. That single rule source leads to generic acclaimed classics (*The Godfather*, *Fargo*, *Pulp Fiction*), and none is a hit. This is the cold-start and long-tail failure mode: when a user's movies are below minsup, association rules have nothing specific to say.

Overall, the recommender works best for users whose history lies in the popular, densely co-liked part of the catalogue, and worst for users with short or niche histories.

9 Conclusions and limitations

- **FP-growth is the right miner for dense movie baskets.** It produced the same itemsets as Apriori in a fraction of the time, and the gap widens as minsup falls.
- **minsup is the knob that matters.** Below about 0.10 the rule count explodes with no gain in recommendation quality. minconf up to 0.5 only prunes redundant long-consequent rules, and above 0.6 it starts to remove the rules that users depend on. The final setting, minsup 0.10 and minconf 0.50, gives 9 281 rules, a number that is still possible to inspect.
- **Ranking by lift beats ranking by confidence**, because confidence rewards popularity rather than association.
- **Limitations.**
  - Only 194 movies can be recommended, which caps recall.
  - Ratings are binarised, so a 3-star and a 5-star rating count the same.
  - Single-antecedent rules ignore combinations of watched movies.

– The test set is small (122 users), so differences of about 0.01 in precision are within noise.

Natural extensions are multi-antecedent rules, per-item adaptive support, and blending rule scores with a popularity prior for users who trigger no rules.

Reproducibility. Running the notebook from top to bottom regenerates every number, figure and both rule files. It downloads the dataset if it is missing and uses seed 42 for all random choices. The full run takes a few minutes on a laptop.